

Christos Mountakis 03801

Elli Zisopoulou 03880

DISTRIBUTED SYSTEMS

RELIABLE AND CAUSAL-TOTAL GROUP COMMUNICATION

This assignment covers the development of a group communication support in the spirit of reliable causal-total multicast.

The system is built upon:

- **Group Management Service (GMS):** A centralized, reliable service that handles membership, tracks process liveness via heartbeats, and notifies members of "Group Views".
- **Reliable Causal-Total Multicast:** A P2P protocol using UDP/IP unicasts to propagate messages. It ensures that every member receives every message exactly once, and in the same relative order, respecting both causality and a global sequence.

FUNCTIONS

- **grp_init**: Establishes a **TCP connection** to the GMS and binding a **UDP socket** for P2P messaging. Launches a background listener thread to handle incoming network traffic and retransmissions.
- **grp_join**: Registers the process with a specific group by sending a request to the GMS. It allocates local resources and waits for the GMS to assign it a unique **PID**.
- **grp_leave**: Removes the process from a group. It notifies the GMS to update the group view and cleans up local memory and synchronization primitives.
- **grp_send**: Sends a message to all members of the group. It assigns a local sequence number for **causal ordering**, buffers the message locally for reliability and then uses **UDP unicasts** to transmit the data to all other group members.
- **grp_recv**: Retrieves the next available message from the local delivery queue.
- It ensures the application only sees messages that have been properly ordered (**total order**) by the protocol.
- It can operate in **blocking mode** (waiting for a message to arrive) or **non-blocking mode**.

STRUCTS

- **PendingEntry**: Tracks a specific message from the moment it is sent/received until it is fully acknowledged by everyone. It stores the payload and an array of which members have confirmed receipt.
- **DlvrNumEntry**: Tracks the reliability of the coordinator's ordering messages (DELIVER_NR) to ensure no member misses an ordering assignment.
- **LocalGroup**: It maintains the "Group View", the TRM state, and the recovery status.
 - *total_delivered*: The counter for the next expected message to be given to the app.
 - *expected_seqnum*: Array used to enforce **causal ordering** from each sender.
 - *is_recovering*: A flag that pauses the group when the coordinator dies.
 - *delivery_queue*: A circular buffer where ordered messages sit until the application calls `grp_rcv()`.
- **ProcessState**: It holds the sockets for TCP and UDP, and an array of LocalGroup structures, allowing one process to be part of multiple groups at once.

TYPES OF MESSAGES

- **MSG**: Data message containing the application-level payload.
- **MSG_ACK**: An acknowledgment sent back to the sender of a MSG. The sender tracks these ACKs in a PendingEntry to know when a message has been successfully received by the entire group.
- **DELIVER_NR**: Sent by the coordinator to assign a global sequence number to a specific MSG.
- **DELIVER_NR_ACK**: Sent by members back to the coordinator to acknowledge they have received the ordering information.
- **COORD_STATE_REQ**: Sent by the new coordinator to all surviving members to learn their state.
- **COORD_STATE_REPLY**: Members respond with their local state.
- **RECOVERY_DONE**: Sent by the new coordinator once it has synthesized the global state and filled any "gaps" in the message sequence.

RELIABLE MULTICAST

- Every message is assigned a **MsgID** consisting of the sender's PID and a local sequence number.
- When a member receives a MSG, it immediately sends a MSG_ACK back to the original sender.
- The sender maintains a PendingEntry for every message it sends, which includes a boolean array **acked_by[MAX_MEMBERS]**.
- A message is only considered "fully acknowledged" once every active member in the current "Group View" has sent an ACK.
- Each **PendingEntry** has a **retransmit_count**. Every time the timer fires, this count increments.
- When the count hits the RETRANSMIT_THRESHOLD, the function iterates through the group members and resends the MSG only to those who haven't acked it yet.

COORDINATOR FAILURE

When the coordinator fails, the system enters a **Recovery Phase** to ensure no messages are lost or delivered out of order.

- All members set *lg->is_recovering* = true to freeze normal operations.
- The new coordinator sends a COORD_STATE_REQ message to all other survivors to gather their state.
- Each member sends back a COORD_STATE_REPLY containing the highest delivery number.
- The new coordinator finds the *recovery_max_dlv* and the *recovery_min_delivered* from the replies to implement:
 - Gap Filling: For any sequence numbers missing a message, it creates "void tickets" to prevent the delivery queue from stalling.
 - Ghost Purging: It removes any "ghost" sequence numbers that were higher than the global *recovery_max_dlv*.
- It broadcasts a RECOVERY_DONE message to all members.
- Finally, *trm_try_deliver* is called to deliver any messages that were held up during the failure.

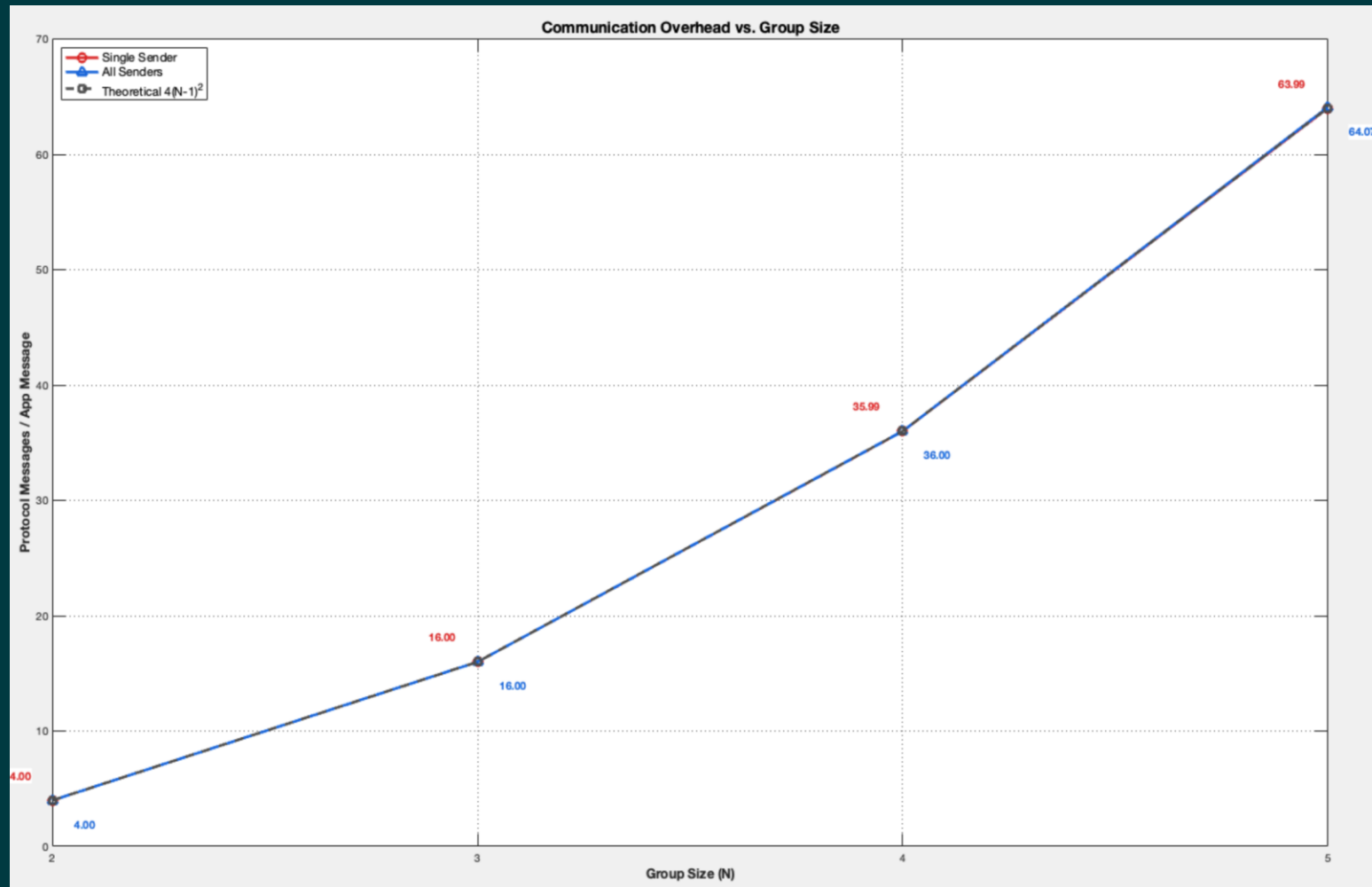
MSG/DELIVERY PHASE

Both msg and delivery phase is as shown.

Total phase = $4(n-1)^2$

$$\begin{aligned} & (N - 1) + (N - 1) + (N - 1)(N - 2) + (N - 1)(N - 2) \\ &= 2(N - 1) + 2(N - 1)(N - 2) \\ &= 2(N - 1)[1 + (N - 2)] \\ &= 2(N - 1)^2 \end{aligned}$$

THEORITICAL VS PRACTICAL



THROUGHPUT VS GROUP SIZE

